

Verifiable Information-Theoretic HSS over General Adversary Structures

Yang Liu^{1,2} and Liang Feng Zhang¹

¹ ShanghaiTech University, Shanghai, China

² Bocconi University, Milan, Italy

yang.liu2@phd.unibocconi.it, zhanglf@shanghaitech.edu.cn

Abstract. Homomorphic secret sharing (HSS) is a cryptographic framework enabling clients to distribute private inputs among multiple servers. These servers subsequently evaluate a function over the shares without compromising the confidentiality of the underlying data. In an additive HSS scheme, the final result is reconstructed by summing the output shares provided by the servers. Although efficient, standard HSS assumes that servers follow the protocol honestly. Verifiable HSS (VHSS) adds a layer of security by requiring servers to prove their computations are correct, protecting against active adversaries that might return modified output shares. However, most existing VHSS schemes are restricted to threshold adversary structures and rely on cryptographic assumptions, lacking information-theoretic security. Moreover, the few existing information-theoretic schemes exhibit significant limitations. They either restrict the adversary to multipartite structures, incur an impractical $O(m^d)$ proof size overhead, or require a stronger Q_{d+1} adversary structure (where no $d+1$ unauthorized subsets cover all the servers) to evaluate degree- d polynomials. This paper addresses these gaps by proposing a generic compiler that transforms any standard additive information-theoretic HSS scheme into a VHSS scheme capable of evaluating full degree- d polynomials. Our compiler supports general adversary structures of type Q_d and achieves a significantly smaller, linear proof size of $O(m)$. To the best of our knowledge, this is the first information-theoretic VHSS scheme for degree- d polynomial evaluation over Q_d adversary structures with linear proof size.

Keywords: Homomorphic Secret Sharing · Information-Theoretic · Verifiable · Polynomial Evaluation · General Adversary Structures.

1 Introduction

Secret sharing (SS) is a fundamental cryptographic primitive protecting private inputs from leakage by dividing them into shares and distributing them among n servers. This ensures predefined authorized subsets of servers can reconstruct the input while unauthorized subsets obtain no information regarding it. Shamir [24] and Blakley [3] introduced *threshold* SS, which requires at least t servers to form an authorized subset. Ito et al. [18] later extended SS to support *general*

access structures, where authorized subsets comprise an arbitrary collection of server subsets. SS schemes have been widely used in various cryptographic and distributed applications such as unconditionally secure multiparty computation (MPC) [1,9,13,17].

Barkol et al. [2] introduced the concept of *d-multiplicative* SS (d-MSS) to address open problems in MPC, particularly secure polynomial evaluation. An SS scheme is termed *d-multiplicative* if servers can locally transform the input shares of d private inputs into additive shares of their product. It was proven that a *d-MSS* scheme exists if and only if the set of all unauthorized subsets, called an *adversary structure*, is of type Q_d . This type guarantees that no d unauthorized subsets can cover all servers. Subsequently, Boyle et al. [5] proposed *homomorphic* SS (HSS), which enables servers to locally evaluate functions on input shares without interaction. In *additive* HSS, the simplest variant, the output shares are additive shares of the function’s output. Consequently, any *d-MSS* scheme can be seen as an HSS scheme for degree- d monomials, and can naturally be extended to an HSS scheme for degree- d polynomials through a linear combination of the monomial output shares.

Upon receiving the output shares from the servers, the client reconstructing the final output must detect any malicious servers that may have transmitted incorrect shares. To this end, several recent works [10,11,12,15,25,26,27,28,29,30,31] focus on the *verifiability* of HSS, termed verifiable HSS (VHSS). This paradigm allows servers to locally generate proofs from their input shares, demonstrating that the output corresponds to the correct function evaluation on the private inputs. However, the majority of existing VHSS schemes [10,11,12,25,26,27,28] *are restricted to threshold adversary structures and lack information-theoretic security*. To the best of our knowledge, the information-theoretic exceptions are the schemes by Yoshida and Obana [29,30,31] and Eriguchi et al. [15]. However, existing frameworks exhibit significant limitations. The schemes in [29] *require a Q_{d+1} adversary structure* to evaluate degree- d polynomials or otherwise only *support a restricted subclass of such polynomials*. Moreover, the schemes in [30,31] exclusively verify monomials, and any trivial extension to full polynomial evaluation incurs an $O(m^d)$ *proof size overhead*, where m denotes the number of inputs. Meanwhile, although the scheme in [15] supports polynomial evaluation, it is specifically *designed for multipartite adversary structures*, which heavily restricts its applicability in general adversary settings.

Therefore, to the best of our knowledge, no prior information-theoretic VHSS construction is known that simultaneously supports degree- d polynomial evaluation over general Q_d adversary structures and achieves proof size at most linear in the number m of input values.

1.1 Our Contribution

We introduce a compiler that converts any additive information-theoretic HSS scheme into an information-theoretic VHSS scheme for polynomial evaluation over general adversary structures. The primary contributions are summarized below.

- **VHSS for degree- d polynomials over Q_d adversary structures.** **Section 3** details a generic compiler that transforms any m -client, n -server additive information-theoretic HSS scheme into an m -client, n -server VHSS scheme. This transformation supports degree- d polynomial evaluation and tolerates a Q_d adversary structure Δ . Each input share consists of the underlying HSS input share and three additional field elements, whereas the output share remains a single field element. The proof requires $3m + 1$ field elements. Although our scheme introduces an adversary success probability of $\frac{m+1}{p}$ that scales linearly with the number of inputs due to the parallel nature of our verification checks, this bound remains practically negligible for sufficiently large cryptographic fields and serves as a worthwhile trade-off to compress proof sizes from $O(m^d)$ down to $O(m)$.

Furthermore, we prove the security of this VHSS scheme against unbounded adversaries capable of corrupting any unauthorized subset of servers defined by the adversary structure, without relying on cryptographic assumptions. To the best of our knowledge, this represents the first information-theoretic VHSS scheme for degree- d polynomials over Q_d adversary structures with linear proof size. Table 1 compares our proposed scheme with existing information-theoretic VHSS schemes that support polynomial evaluation.

Table 1: Comparison of Information-Theoretic VHSS For Polynomial Evaluation

Scheme	Δ^1	Generic Construction	Proof Size ²	Function Class ³
Thm. 4 of [29]	Q_{d+1}	✓	$O(1)$	$\mathcal{P}_d$
[15]	$Q_{d+1}^{(\ell)}$	✗	$O(n)$	$\mathcal{P}_d$
Thm. 5 of [29]	Q_d	✓	$O(1)$	$\mathcal{P}_d^*$
[30]	Q_d	✓	$O(m^d)$	$\mathcal{P}_d$
[31]	Q_d	✓	$O(m^d)$	$\mathcal{P}_d$
Our Scheme	Q_d	✓	$O(m)$	$\mathcal{P}_d$

¹ Type of adversary structures, where $Q_{d+1}^{(\ell)} \subseteq Q_{d+1} \subseteq Q_d$. The notation $Q_{d+1}^{(\ell)}$ indicates the ℓ -partite Q_{d+1} adversary structure, where servers are partitioned into ℓ subsets such that servers within the same subset assume identical roles. See [15] for details.

² Here, m denotes the number of clients, n denotes the number of servers, and d represents the degree of the polynomial.

³ $\mathcal{P}_d$ denotes the class of degree- d polynomials, and $\mathcal{P}_d^*$ denotes the class of degree- d polynomials that can be represented by $x_i \cdot f_1 + f_2$ for some variable x_i and polynomials f_1 and f_2 of degree at most $d - 1$. It can be observed that $\mathcal{P}_d^* \subsetneq \mathcal{P}_d$.

1.2 Related Work

The notion of VHSS was formally introduced by Tsaloli et al. [26]. However, prior to [26], VHSS schemes for specific functions such as the *monomials*, the *selection functions* and the *point functions* had been implicitly constructed in the study of d -multiplicative secret sharing [30], private information retrieval [32] and function secret sharing schemes [6], respectively.

A distinct line of research [10,11,12,25,26,27,28] focuses on the construction of VHSS schemes *for the threshold adversary structures*. To the best of our knowledge, these schemes rely either on computational assumptions, such as fixed inversion problem [25,26], discrete logarithm problem [27] and learning with errors problem [10,11,28] or based on the existence of homomorphic encryption schemes [12]; consequently, they do not provide information-theoretic security against unbounded adversaries.

Yoshida and Obana [29] introduced a compiler that transforms an additive HSS scheme into a VHSS scheme, establishing the first information-theoretic VHSS scheme compatible with general adversary structures. However, their approach *either requires a Q_{d+1} adversary structure to evaluate degree- d polynomials or only supports a restricted subclass of degree- d polynomials*. Subsequent research [30,31] enabled the verification of degree- d monomials over a Q_d adversary structure. Nevertheless, *trivially extending these methods to full polynomial evaluation incurs an $O(m^d)$ proof size overhead*. Eriguchi et al. [15] proposed an HSS scheme for polynomial evaluation supporting multipartite adversary structures, which they extended into a VHSS scheme. This method, however, *cannot support general adversary structures*. Furthermore, its approach to verifiability relies on the specific properties of multipartite structures, and therefore *cannot be easily adapted to other HSS schemes*.

Beyond the VHSS literature, our construction shares conceptual similarities with authenticated secret-sharing protocols such as SPDZ [14,19], which employ MACs to enable verifiable computation over shared data. In our scheme, quadratic checks added during the sharing phase serve a similar verification purpose. However, SPDZ requires communication for every multiplication, relies on preprocessing and computational assumptions, and tolerates any number of corrupted servers as long as at least one remains honest, whereas our compiler allows servers to evaluate polynomials locally without interaction, is preprocessing-free, achieves information-theoretic security, and requires the Q_d structure. We emphasize that the verification technique used here is inherently tied to the additive and homomorphic properties of HSS, and extending it to general MPC is left as future work.

2 Preliminaries

2.1 Notations

Let $\mathbb{F}_p$ denote the finite field of order p , where p is an odd prime power; equivalently, $\mathbb{F}_p$ has odd characteristic. We denote the set of m -variate polynomials

over $\mathbb{F}_p$ by $\mathbb{F}_p[X_1, \dots, X_m]$. For any $f \in \mathbb{F}_p[X_1, \dots, X_m]$, let $\deg(f)$ denote the degree of f , defined as the maximum degree of any monomial in f . For a finite set S , the notation $s \leftarrow S$ represents sampling an element s uniformly at random from S . For any integer $n > 0$, let $[n] = \{1, \dots, n\}$. The power set of S is denoted by 2^S , and S^m represents the m -fold Cartesian product of S . For a positive integer m , let $\mathbf{s} = (s_1, \dots, s_m)$ denote a vector in S^m .

2.2 Adversary Structure

We recall the notion of adversary structure [2], which specifies a collection of *unauthorized* sets.

Definition 1 (Adversary Structure). *An n -server adversary structure is a collection of sets $\Delta \subseteq 2^{[n]}$ that is closed under subsets. Formally, if $A \in \Delta$ and $B \subseteq A$, then $B \in \Delta$.*

We assume that Δ contains a non-empty subset of $[n]$. This assumption is justified because if $\Delta = \{\emptyset\}$, every non-empty subset would be authorized, implying that any single server could recover the secret.

Definition 2 (Adversary Structure of Type Q_d [2]). *Let n and d be positive integers, and let Δ be an n -server adversary structure. The adversary structure Δ satisfies the Q_d property if, for any d sets $T_1, \dots, T_d \in \Delta$, the union $T_1 \cup \dots \cup T_d \neq [n]$. In other words, no collection of d unauthorized sets can cover the entire set of servers.*

2.3 Homomorphic Secret Sharing

We recall the notation of homomorphic secret sharing (HSS) [7,20]. The definitions presented here pertain to the *information-theoretic setting* and focus on the evaluation of polynomials.

Definition 3 (Homomorphic Secret Sharing). *An (information-theoretic) m -client n -server homomorphic secret sharing scheme for degree- d polynomials over $\mathbb{F}_p$, denoted by $\text{HSS} = (\text{SHARE}, \text{EVAL}, \text{DEC})$, comprises the following algorithms.*

- $(\text{in}_i^{(j)})_{i \in [n]} \leftarrow \text{SHARE}(j, x^{(j)})$: On input of a client index $j \in [m]$ and a value $x^{(j)} \in \mathbb{F}_p$, the sharing algorithm outputs a tuple of input shares $(\text{in}_i^{(j)})_{i \in [n]}$, where $\text{in}_i^{(j)} \in \{0, 1\}^\alpha$.
- $\text{out}_i \leftarrow \text{EVAL}(i, f, (\text{in}_i^{(j)})_{j \in [m]})$: On input of a server index $i \in [n]$, a description of an m -variate polynomial $f \in \mathbb{F}_p[X_1, \dots, X_m]$ with $\deg(f) \leq d$, and a set of m input shares $(\text{in}_i^{(j)})_{j \in [m]}$, the evaluation algorithm outputs a share $\text{out}_i \in \{0, 1\}^\beta$.
- $y \leftarrow \text{DEC}((\text{out}_i)_{i \in [n]})$: On input of the output shares $(\text{out}_i)_{i \in [n]}$, the decoding algorithm reconstructs the final output $y \in \mathbb{F}_p$.

The parameters $\alpha = \alpha(n)$ and $\beta = \beta(m, n)$ denote the bit-lengths of the input and output shares, respectively.

Definition 4 (Correctness of HSS). An m -client n -server HSS scheme for degree- d polynomials over $\mathbb{F}_p$ is correct if for any polynomial $f \in \mathbb{F}_p[X_1, \dots, X_m]$ with $\deg(f) \leq d$, any inputs $x^{(1)}, \dots, x^{(m)} \in \mathbb{F}_p$, any input shares $(\text{in}_i^{(j)})_{i \in [n]} \leftarrow \text{SHARE}(j, x^{(j)})$ for $j \in [m]$, and any outputs $\text{out}_i \leftarrow \text{EVAL}(i, f, (\text{in}_i^{(j)})_{j \in [m]})$ for $i \in [n]$, it holds that

$$\Pr \left[\text{DEC}((\text{out}_i)_{i \in [n]}) = f(x^{(1)}, \dots, x^{(m)}) \right] = 1.$$

Definition 5 (Δ -privacy of HSS). Let Δ be an n -server adversary structure. An m -client n -server HSS scheme HSS satisfies Δ -privacy if, for any subset $T \in \Delta$, any distinct client $j_1, j_2 \in [m]$, and any inputs $x^{(j_1)}, x^{(j_2)} \in \mathbb{F}_p$, the following distributions

$$\{(\text{in}_i^{(j_1)})_{i \in T} : (\text{in}_i^{(j_1)})_{i \in [n]} \leftarrow \text{SHARE}(j_1, x^{(j_1)})\}$$

and

$$\{(\text{in}_i^{(j_2)})_{i \in T} : (\text{in}_i^{(j_2)})_{i \in [n]} \leftarrow \text{SHARE}(j_2, x^{(j_2)})\}$$

are identical.

We use the notation (m, n, d, Δ) -HSS to denote an m -client n -server HSS scheme for degree- d polynomials over $\mathbb{F}_p$ that satisfies correctness and Δ -privacy.

Definition 6 (Additive HSS [7]). An (m, n, d, Δ) -HSS scheme over $\mathbb{F}_p$ is additive if, for every polynomial $f \in \mathbb{F}_p[X_1, \dots, X_m]$ satisfying $\deg(f) \leq d$, inputs $x^{(1)}, \dots, x^{(m)} \in \mathbb{F}_p$, shares $(\text{in}_i^{(j)})_{j \in [m], i \in [n]}$, and server index $i \in [n]$, the following conditions apply.

1. The evaluation algorithm EVAL yields a share $\text{out}_i \in \mathbb{F}_p$.
2. The decoding algorithm DEC computes the result as $y = \sum_{i \in [n]} \text{out}_i$.

Theorem 4.6 in [2] establishes that for any adversary structure Δ , a d -MSS scheme exists if and only if Δ is of type Q_d . Given that d -MSS is equivalent to additive HSS for degree- d polynomials³, we obtain the following property for additive HSS schemes.

Proposition 1. For any positive integers m, n, d and an n -server adversary structure Δ , an additive (m, n, d, Δ) -HSS scheme exists if and only if Δ is of type Q_d .

³ A d -MSS scheme transforms into an additive HSS scheme for degree- d polynomials by having each server locally evaluate the monomials of the target polynomial and sum the results.

2.4 Verifiable Homomorphic Secret Sharing

The verifiable HSS (VHSS) scheme [26] extends the HSS framework by introducing *verifiability*. In an m -client n -server VHSS scheme, each server i computes an output share out_i via the EVAL algorithm and a proof share pf_i using a PROOF algorithm. The VER algorithm subsequently utilizes the set of proof shares $\{\text{pf}_i\}_{i \in [n]}$ to verify that the reconstructed value $y = \text{DEC}(\{\text{out}_i\}_{i \in [n]})$ corresponds to the correct evaluation of the polynomial f on the provided inputs.

It is important to note that throughout this work, we operate under the *honest-client assumption*. We assume that all clients correctly execute the SHARE algorithm and distribute well-formed, consistent input shares to the servers. Considering malicious clients who might distribute inconsistent shares significantly complicates the verification process, as a standard MAC check may no longer suffice. We consider the malicious client setting to be outside the scope of this paper.

Definition 7 (Verifiable HSS). *An (information-theoretic) m -client n -server verifiable HSS scheme for degree- d polynomials over $\mathbb{F}_p$, denoted by $\text{VHSS} = (\text{SHARE}, \text{EVAL}, \text{PROOF}, \text{DEC}, \text{VER})^2$, consists of the following algorithms.*

- $(\text{in}_i^{(j)})_{i \in [n]} \leftarrow \text{SHARE}(j, x^{(j)})$: On input of a client index $j \in [m]$ and an input $x^{(j)} \in \mathbb{F}_p$, the sharing algorithm outputs a tuple of input shares $(\text{in}_i^{(j)})_{i \in [n]}$, where $\text{in}_i^{(j)} \in \{0, 1\}^\alpha$.
- $\text{out}_i \leftarrow \text{EVAL}(i, f, (\text{in}_i^{(j)})_{j \in [m]})$: On input of a server index $i \in [n]$, a description of an m -variate polynomial $f \in \mathbb{F}_p[X_1, \dots, X_m]$ with $\deg(f) \leq d$, and a set of m input shares $(\text{in}_i^{(j)})_{j \in [m]}$, the evaluation algorithm outputs a share $\text{out}_i \in \{0, 1\}^\beta$.
- $\text{pf}_i \leftarrow \text{PROOF}(i, f, (\text{in}_i^{(j)})_{j \in [m]})$: On input of a server index $i \in [n]$, a description of an m -variate polynomial $f \in \mathbb{F}_p[X_1, \dots, X_m]$ with $\deg(f) \leq d$, and a set of m input shares $(\text{in}_i^{(j)})_{j \in [m]}$, the proof algorithm outputs a proof share $\text{pf}_i \in \{0, 1\}^\gamma$.
- $y \leftarrow \text{DEC}((\text{out}_i)_{i \in [n]})$: On input of the output shares $(\text{out}_i)_{i \in [n]}$, the decoding algorithm reconstructs the final output $y \in \mathbb{F}_p$.
- $b \leftarrow \text{VER}(y, (\text{pf}_i)_{i \in [n]})$: On input of the candidate output y and the set of proof shares $(\text{pf}_i)_{i \in [n]}$, the verification algorithm outputs a bit $b \in \{0, 1\}$.

The parameters $\alpha = \alpha(n)$, $\beta = \beta(m, n)$, and $\gamma = \gamma(m, n)$ denote the bit-lengths of the input, output, and proof shares, respectively. A proof share is termed compact if γ is independent of n and m .

Definition 8 (Correctness of VHSS). *An m -client n -server VHSS scheme VHSS for degree- d polynomials over $\mathbb{F}_p$ is correct if for any $f \in \mathbb{F}_p[X_1, \dots, X_m]$ with $\deg(f) \leq d$, any inputs $x^{(1)}, \dots, x^{(m)} \in \mathbb{F}_p$, any input shares $(\text{in}_i^{(j)})_{i \in [n]} \leftarrow$*

² Although certain prior VHSS schemes [26,30] employ a distinct decoding algorithm for proof shares, our model integrates this step into the VER algorithm because the output client executes both procedures.

$\text{SHARE}(j, x^{(j)})$ for each $j \in [m]$, any output shares $\text{out}_i \leftarrow \text{EVAL}(i, f, (\text{in}_i^{(j)})_{j \in [m]})$, any proof shares $\text{pf}_i \leftarrow \text{PROOF}(i, f, (\text{in}_i^{(j)})_{j \in [m]})$ for $i \in [n]$, any output $y \leftarrow \text{DEC}((\text{out}_i)_{i \in [n]})$, it holds that

$$\Pr [\text{VER}(y, (\text{pf}_i)_{i \in [n]}) = 1] = 1.$$

Definition 9 (Δ -privacy of VHSS). Let Δ be an n -server adversary structure. An m -client n -server VHSS scheme VHSS satisfies Δ -privacy if, for any subset $T \in \Delta$, any distinct client $j_1, j_2 \in [m]$, and any inputs $x^{(j_1)}, x^{(j_2)} \in \mathbb{F}_p$, the following distributions

$$\{(\text{in}_i^{(j_1)})_{i \in T} : (\text{in}_i^{(j_1)})_{i \in [n]} \leftarrow \text{SHARE}(j_1, x^{(j_1)})\}$$

and

$$\{(\text{in}_i^{(j_2)})_{i \in T} : (\text{in}_i^{(j_2)})_{i \in [n]} \leftarrow \text{SHARE}(j_2, x^{(j_2)})\}$$

are identical.

$\text{Exp}(f, T, \text{Adv})$	
1. For each $j \in [m]$, sample $x^{(j)} \leftarrow \mathbb{F}_p$ and generate input shares	$(\text{in}_i^{(j)})_{i \in [n]} \leftarrow \text{SHARE}(j, x^{(j)}).$
2. For each $i \in T$, let $(\widehat{\text{out}}_i, \widehat{\text{pf}}_i)$ denote the modified output and proof shares generated by the adversary $\text{Adv}(f, (\text{in}_i^{(j)})_{i \in T, j \in [m]})$.	
3. For each $i \notin T$, compute	$\widehat{\text{out}}_i \leftarrow \text{EVAL}(i, f, (\text{in}_i^{(j)})_{j \in [m]})$
and	
	$\widehat{\text{pf}}_i \leftarrow \text{PROOF}(i, f, (\text{in}_i^{(j)})_{j \in [m]}).$
4. Reconstruct $\hat{y} \leftarrow \text{DEC}((\widehat{\text{out}}_i)_{i \in [n]}).$	
5. If	$\hat{y} \neq f(x^{(1)}, \dots, x^{(m)})$
and	
	$\text{VER}(\hat{y}, (\widehat{\text{pf}}_i)_{i \in [n]}) = 1,$
output 1; otherwise, output 0.	

Fig. 1: The verifiability experiment for VHSS.

Definition 10 (ϵ -verifiability of VHSS). Let Δ be an n -server adversary structure. An m -client n -server VHSS scheme VHSS satisfies ϵ -verifiability if

for any $f \in \mathbb{F}_p[X_1, \dots, X_m]$ with $\deg(f) \leq d$, any $T \in \Delta$, and an unbounded adversary Adv who modifies output and proof shares of servers in T , it holds that

$$\Pr[\text{Exp}(f, T, \text{Adv}) = 1] \leq \epsilon,$$

where the experiment $\text{Exp}(f, T, \text{Adv})$ is defined in Fig. 1.

Similarly, we use the notation $(m, n, d, \Delta, \epsilon)$ -VHSS to denote an m -client n -server VHSS scheme for degree- d polynomials satisfying correctness, Δ -privacy, and ϵ -verifiability.

Compared with representative VHSS definitions [15,25,26,30], which consider fixed/arbitrary client inputs, the experiment in Fig. 1 samples each input uniformly at random and therefore does not directly cover adversarially fixed or highly predictable inputs. Nevertheless, random-input security has also been considered in related cryptographic primitives [8]. Moreover, fresh and correlated randomness remains a central resource in modern MPC, from standard preprocessing [14,19] and recent Sumcheck/PCG-based techniques [4,21] to degree- D correlations and actively secure evaluation of general multivariate polynomials [16,23]. Hence, the modified experiment still captures a meaningful cryptographic regime involving fresh randomness and higher-degree polynomial computations.

3 VHSS for degree- d polynomials over Q_d -Adversary Structures

This section presents a compiler that transforms any additive (m, n, d, Δ) -HSS scheme over $\mathbb{F}_p$ into an $(m, n, d, \Delta, \frac{m+1}{p})$ -VHSS scheme over $\mathbb{F}_p$, where Δ satisfies the Q_d property.

3.1 Intuition

The proposed compiler builds upon the following polynomial decomposition lemma.

Lemma 1 (Polynomial Decomposition [22]). *Let $f \in \mathbb{F}_p[X_1, \dots, X_m]$ be an m -variate polynomial. For any set of constants $\theta^{(1)}, \dots, \theta^{(m)} \in \mathbb{F}_p$, there exist polynomials $q_j \in \mathbb{F}_p[X_1, \dots, X_m]$ such that*

$$f(X_1, \dots, X_m) - f(\theta^{(1)}, \dots, \theta^{(m)}) = \sum_{j=1}^m (X_j - \theta^{(j)}) \cdot q_j(X_1, \dots, X_m).$$

Furthermore, the polynomials $(q_j)_{j \in [m]}$ can be computed efficiently. If $\deg(f) \leq d$, they can be chosen so that $\deg(q_j) \leq d - 1$ for every $j \in [m]$.

Let $f \in \mathbb{F}_p[X_1, \dots, X_m]$ denote the target polynomial and $x^{(1)}, \dots, x^{(m)}$ the corresponding inputs. The proof algorithm **PROOF** employs **Lemma 1** with respect to public points $\theta^{(j)} = x^{(j)} - \eta^{(j)}$ to output $f(\theta^{(1)}, \dots, \theta^{(m)})$ and shares of $q_j(x^{(1)}, \dots, x^{(m)})$, thereby enabling verification of the result $y = f(x^{(1)}, \dots, x^{(m)})$ through the identity

$$y - f(\theta^{(1)}, \dots, \theta^{(m)}) = \sum_{j=1}^m \eta^{(j)} \cdot q_j(x^{(1)}, \dots, x^{(m)}),$$

where $\eta^{(j)}$ is selected uniformly at random by client j .

3.2 The Compiler

Let $\text{HSS} = (\text{SHARE}', \text{EVAL}', \text{DEC}')$ denote an additive (m, n, d, Δ) -HSS scheme with efficiency parameters $(\alpha', \beta', \gamma')$. Let $x^{(1)}, \dots, x^{(m)} \in \mathbb{F}_p$ be m arbitrary inputs. Let $f \in \mathbb{F}_p[X_1, \dots, X_m]$ be a polynomial with $\deg(f) \leq d$. We construct an $(m, n, d, \Delta, \frac{m+1}{p})$ -VHSS scheme over $\mathbb{F}_p$, denoted as $\text{VHSS} = (\text{SHARE}, \text{EVAL}, \text{PROOF}, \text{DEC}, \text{VER})$, detailed in Fig. 2.

The sharing algorithm **SHARE** distributes the input $x^{(j)}$ of each client $j \in [m]$ among the servers using the underlying algorithm SHARE' , augmented with random masks for verification. Specifically, client j samples $\eta^{(j)} \leftarrow \mathbb{F}_p$ and computes $\delta^{(j)} = (\eta^{(j)})^2$. These values are additively shared among the servers as $(\eta_i^{(j)})_{i \in [n]}$ and $(\delta_i^{(j)})_{i \in [n]}$. Client j also generates $(u_i^{(j)})_{i \in [n]} \leftarrow \text{SHARE}'(j, x^{(j)})$ and computes the masked input $\theta^{(j)} = x^{(j)} - \eta^{(j)}$. The resulting input share for server i associated with client j is $\text{in}_i^{(j)} = (u_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)}, \theta^{(j)})$.

The evaluation algorithm **EVAL** computes the output share for each server $i \in [n]$ by applying the underlying HSS evaluation algorithm to the u -components of the input shares. Server i parses each input share $\text{in}_i^{(j)}$ as $(u_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)}, \theta^{(j)})$ and computes $\text{out}_i \leftarrow \text{EVAL}'(i, f, u_i^{(1)}, \dots, u_i^{(m)})$. Following the correctness of the underlying HSS scheme, the aggregated output shares yield $\sum_{i \in [n]} \text{out}_i = f(x^{(1)}, \dots, x^{(m)})$.

The proof algorithm **PROOF** generates the proof share pf_i by first evaluating the target polynomial f on the masked inputs $(\theta^{(1)}, \dots, \theta^{(m)})$, i.e., each server i computes $\sigma_i = f(\theta^{(1)}, \dots, \theta^{(m)})$. Subsequently, the server i determines polynomials $q_1, \dots, q_m \in \mathbb{F}_p[X_1, \dots, X_m]$ such that the following identity holds:

$$f(X_1, \dots, X_m) - \sigma_i = \sum_{j=1}^m (X_j - \theta^{(j)}) \cdot q_j(X_1, \dots, X_m).$$

For each $j \in [m]$, the server i computes $\tau_i^{(j)} \leftarrow \text{EVAL}'(i, q_j, (u_i^{(k)})_{k \in [m]})$. The final proof share of server i is $\text{pf}_i = (\sigma_i, (\tau_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)})_{j \in [m]})$, where $\eta_i^{(j)}, \delta_i^{(j)}$ are derived from $\text{in}_i^{(j)}$.

The decoding algorithm **DEC** reconstructs the final computation result y by summing the respective output shares, yielding $y = \sum_{i \in [n]} \text{out}_i$.

The verification algorithm **VER** confirms the integrity of the masks and the correctness of the polynomial evaluation. For each $j \in [m]$, it first reconstructs

SHARE $(j, x^{(j)})$:

1. Sample $\eta^{(j)} \leftarrow \mathbb{F}_p$ and let $\delta^{(j)} = (\eta^{(j)})^2$. Sample $(\eta_i^{(j)})_{i \in [n]}, (\delta_i^{(j)})_{i \in [n]} \leftarrow \mathbb{F}_p$ such that
$$\eta^{(j)} = \sum_{i \in [n]} \eta_i^{(j)} \quad \text{and} \quad \delta^{(j)} = \sum_{i \in [n]} \delta_i^{(j)}.$$
2. Let $(u_i^{(j)})_{i \in [n]} \leftarrow \text{SHARE}'(j, x^{(j)})$ and $\theta^{(j)} = x^{(j)} - \eta^{(j)}$.
3. Return $(\text{in}_i^{(j)})_{i \in [n]}$, where $\text{in}_i^{(j)} = (u_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)}, \theta^{(j)})$.

EVAL $(i, f, (\text{in}_i^{(j)})_{j \in [m]})$:

1. For each $j \in [m]$, parse $\text{in}_i^{(j)} = (u_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)}, \theta^{(j)})$.
2. Return $\text{out}_i \leftarrow \text{EVAL}'(i, f, u_i^{(1)}, \dots, u_i^{(m)})$.

PROOF $(i, f, (\text{in}_i^{(j)})_{j \in [m]})$:

1. For each $j \in [m]$, parse $\text{in}_i^{(j)} = (u_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)}, \theta^{(j)})$.
2. Compute $\sigma_i = f(\theta^{(1)}, \dots, \theta^{(m)})$ and determine polynomials $q_1, \dots, q_m \in \mathbb{F}_p[X_1, \dots, X_m]$ satisfying
$$f(X_1, \dots, X_m) - \sigma_i = \sum_{j=1}^m (X_j - \theta^{(j)}) \cdot q_j(X_1, \dots, X_m).$$
3. Compute $\tau_i^{(j)} \leftarrow \text{EVAL}'(i, q_j, (u_i^{(k)})_{k \in [m]})$ for each $j \in [m]$.
4. Return $\text{pf}_i = (\sigma_i, (\tau_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)})_{j \in [m]})$.

DEC $((\text{out}_i)_{i \in [n]})$:

1. Return $y = \sum_{i \in [n]} \text{out}_i$.

VER $(y, (\text{pf}_i)_{i \in [n]})$:

1. For each $i \in [n]$, parse pf_i as $\text{pf}_i = (\sigma_i, (\tau_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)})_{j \in [m]})$.
2. For each $j \in [m]$, reconstruct $\eta^{(j)} = \sum_{i \in [n]} \eta_i^{(j)}$ and $\delta^{(j)} = \sum_{i \in [n]} \delta_i^{(j)}$, and check that
$$(\eta^{(j)})^2 = \delta^{(j)}.$$

If the check fails for any j , return 0.
3. Reconstruct $\tau^{(j)} = \sum_{i \in [n]} \tau_i^{(j)}$ for each $j \in [m]$.
4. Return 1 if
$$y - \sigma_i = \sum_{j=1}^m \eta^{(j)} \cdot \tau^{(j)}$$

holds for all $i \in [n]$; otherwise, return 0.

Fig. 2: Compiler from additive (m, n, d, Δ) -HSS to $(m, n, d, \Delta, \frac{m+1}{p})$ -VHSS.

the two separately shared values

$$\eta^{(j)} = \sum_{i \in [n]} \eta_i^{(j)} \quad \text{and} \quad \delta^{(j)} = \sum_{i \in [n]} \delta_i^{(j)},$$

and then checks

$$(\eta^{(j)})^2 = \delta^{(j)}.$$

If any check fails, the algorithm outputs 0. Otherwise, it reconstructs $\tau^{(j)} = \sum_{i \in [n]} \tau_i^{(j)}$ for each $j \in [m]$ and verifies that

$$y - \sigma_i = \sum_{j=1}^m \eta^{(j)} \cdot \tau^{(j)}$$

holds for all $i \in [n]$. If all checks are passed, the VER algorithm returns 1; otherwise, it returns 0.

3.3 Security Analysis

Theorem 1. *Assume that $\mathbb{F}_p$ has odd characteristic, the n -server adversary structure Δ satisfies the Q_d property and let HSS be an additive (m, n, d, Δ) -HSS over $\mathbb{F}_p$. Then, the algorithms presented in Fig. 2 constitute an $(m, n, d, \Delta, \frac{m+1}{p})$ -VHSS over $\mathbb{F}_p$. Furthermore, given the efficiency parameters (α', β') for HSS, the derived VHSS achieves parameters $(\alpha, \beta, \gamma) = (\alpha' + 3 \log p, \beta', m\beta' + (2m + 1) \cdot \log p)$.*

Proof. We prove the correctness, Δ -privacy, and $\frac{m+1}{p}$ -verifiability of VHSS in Fig. 2, and the correctness of the efficiency parameters in the following.

Correctness. The correctness of VHSS is already proven in-line in the above discussion.

Δ -privacy. The Δ -privacy of the VHSS construction follows directly from the Δ -privacy of the underlying HSS scheme and additive secret sharing. Each input share $\text{in}_i^{(j)}$ comprises the HSS shares, generated as

$$(u_i^{(j)})_{i \in [n]} \leftarrow \text{SHARE}'(j, x^{(j)}),$$

alongside the additive shares $(\eta_i^{(j)}, \delta_i^{(j)})$ of the values $\eta^{(j)}$ and $\delta^{(j)} = (\eta^{(j)})^2$. For any unauthorized set $T \in \Delta$ and inputs $x^{(j_1)}, x^{(j_2)}$ with distinct indices $j_1, j_2 \in [m]$, the Δ -privacy of the HSS scheme ensures that the distributions

$$\{(u_i^{(j_1)})_{i \in T}\} \quad \text{and} \quad \{(u_i^{(j_2)})_{i \in T}\}$$

are identical. Furthermore, because the values $\eta^{(j)}$ and $\delta^{(j)}$ are additively shared among the servers, the distributions of the mask shares held by T are fundamentally independent of the inputs. Thus, the joint distributions

$$\{\text{in}_i^{(j_1)}\}_{i \in T} \quad \text{and} \quad \{\text{in}_i^{(j_2)}\}_{i \in T}$$

are identical, establishing the Δ -privacy of the VHSS scheme.

$\frac{m+1}{p}$ -verifiability. We establish the $\frac{m+1}{p}$ -verifiability of VHSS. Let $(\text{out}_i)_{i \in [n]}$ and $(\text{pf}_i)_{i \in [n]}$ denote the honest output and proof shares, where

$$\text{pf}_i = (\sigma_i, (\tau_i^{(j)}, \eta_i^{(j)}, \delta_i^{(j)})_{j \in [m]}).$$

Similarly, let $(\widehat{\text{out}}_i)_{i \in [n]}$ and $(\widehat{\text{pf}}_i)_{i \in [n]}$ be the shares output by the adversary in the experiment **Exp** (Fig. 1), where

$$\widehat{\text{pf}}_i = (\hat{\sigma}_i, (\hat{\tau}_i^{(j)}, \hat{\eta}_i^{(j)}, \hat{\delta}_i^{(j)})_{j \in [m]}).$$

For the honest execution, we define

$$y = \sum_{i \in [n]} \text{out}_i, \quad \tau^{(j)} = \sum_{i \in [n]} \tau_i^{(j)}, \quad \eta^{(j)} = \sum_{i \in [n]} \eta_i^{(j)}, \quad \delta^{(j)} = \sum_{i \in [n]} \delta_i^{(j)},$$

and analogously for $\hat{y}, \hat{\tau}^{(j)}, \hat{\eta}^{(j)}, \hat{\delta}^{(j)}$, where

$$\hat{y} = \sum_{i \in [n]} \widehat{\text{out}}_i, \quad \hat{\tau}^{(j)} = \sum_{i \in [n]} \hat{\tau}_i^{(j)}, \quad \hat{\eta}^{(j)} = \sum_{i \in [n]} \hat{\eta}_i^{(j)}, \quad \hat{\delta}^{(j)} = \sum_{i \in [n]} \hat{\delta}_i^{(j)}.$$

We represent the adversary's influence on the reconstructed values by additive modifications $\xi_y, \xi_{\tau,j}, \xi_{\eta,j}, \xi_{\delta,j}$, and $\xi_{\sigma,i}$, writing

$$\begin{aligned} \hat{y} &= y + \xi_y, & \hat{\tau}^{(j)} &= \tau^{(j)} + \xi_{\tau,j}, & \hat{\eta}^{(j)} &= \eta^{(j)} + \xi_{\eta,j}, \\ \hat{\delta}^{(j)} &= \delta^{(j)} + \xi_{\delta,j}, & \hat{\sigma}_i &= \sigma_i + \xi_{\sigma,i}. \end{aligned}$$

These quantities describe the additive modifications introduced by the adversary. In the conditioning argument below, once the adversary's view and random coins are fixed, the ξ -values are fixed, whereas the hidden mask values $(\eta^{(j)})_{j \in [m]}$ remain uniformly distributed. The experiment returns 1 (signifying a successful forgery) only when the adversary causes a non-zero error in the output ($\xi_y \neq 0$) while the **VER** algorithm output remains 1. As shown in Fig. 2, the **VER** algorithm can be divided into two parts:

1. For each $j \in [m]$, the algorithm checks if the reconstructed values satisfy the quadratic relation

$$(\hat{\eta}^{(j)})^2 = \hat{\delta}^{(j)}.$$

2. For each $i \in [n]$, the algorithm checks if the reconstructed values satisfy the linear relation

$$\hat{y} - \hat{\sigma}_i = \sum_{j=1}^m \hat{\eta}^{(j)} \cdot \hat{\tau}^{(j)}.$$

The algorithm outputs 1 if all checks are passed; otherwise, it outputs 0. Let E_j be the event that the adversary succeeds in modifying $\eta^{(j)}$ ($\xi_{\eta,j} \neq 0$) yet passes

the first check. It's important to note that the event E_j occurs if and only if the adversary's modifications to $\eta^{(j)}$ and $\delta^{(j)}$ satisfy the relation

$$(\hat{\eta}^{(j)})^2 = \hat{\delta}^{(j)}.$$

Substituting the above relation into the honest relation

$$(\eta^{(j)})^2 = \delta^{(j)}$$

yields the condition

$$(\xi_{\eta,j})^2 + 2 \cdot \eta^{(j)} \cdot \xi_{\eta,j} - \xi_{\delta,j} = 0.$$

Since only the servers in T may alter their shares, these additive modifications are induced entirely by the changes to shares held by servers in T ; in particular,

$$\xi_{\eta,j} = \sum_{i \in T} (\hat{\eta}_i^{(j)} - \eta_i^{(j)}) \quad \text{and} \quad \xi_{\delta,j} = \sum_{i \in T} (\hat{\delta}_i^{(j)} - \delta_i^{(j)}).$$

By Step 1 of the verifiability experiment, the pairs $(x^{(j)}, \eta^{(j)})_{j \in [m]}$ are sampled independently and uniformly, while the adversary observes $\theta^{(j)} = x^{(j)} - \eta^{(j)}$ for every j . For every fixed vector $(\theta^{(j)})_{j \in [m]}$, the conditional distribution of $(\eta^{(j)})_{j \in [m]}$ remains uniform over $\mathbb{F}_p^m$. Moreover, by the Δ -privacy of the underlying HSS and the privacy of the additive sharings held by the proper corrupted set T , the remaining components of the adversarial view do not reveal this mask vector. Thus, after conditioning on the adversary's view and random coins, the quantities $\xi_{\eta,j}$ and $\xi_{\delta,j}$ are fixed with respect to $\eta^{(j)}$, whereas $\eta^{(j)}$ remains uniform over $\mathbb{F}_p$. Consequently, whenever $\xi_{\eta,j} \neq 0$, the expression

$$(\xi_{\eta,j})^2 + 2 \cdot \eta^{(j)} \cdot \xi_{\eta,j} - \xi_{\delta,j}$$

is a nonzero polynomial of degree one in $\eta^{(j)}$. Indeed, since $\mathbb{F}_p$ has odd characteristic, its linear coefficient $2\xi_{\eta,j}$ is nonzero. Hence the polynomial has at most one root in $\mathbb{F}_p$, and therefore

$$\Pr[E_j] = \Pr \left[\xi_{\eta,j} \neq 0 \wedge (\xi_{\eta,j})^2 + 2 \cdot \eta^{(j)} \cdot \xi_{\eta,j} - \xi_{\delta,j} = 0 \right] \leq \frac{1}{p},$$

after averaging over the adversary's view and random coins. Let E be the event that $\exists j \in [m]$ such that E_j occurs. By the union bound,

$$\Pr[E] \leq \sum_{j=1}^m \Pr[E_j] \leq \frac{m}{p}.$$

For the adversary to modify y ($\xi_y \neq 0$) while passing the second check, its modifications to y , σ_i , $\eta^{(j)}$, and $\tau^{(j)}$ must satisfy the relation

$$\hat{y} - \hat{\sigma}_i = \sum_{j=1}^m \hat{\eta}^{(j)} \cdot \hat{\tau}^{(j)}.$$

Since Δ is of type Q_d , every $T \in \Delta$ is a proper subset of $[n]$, so at least one server is honest. Since all honest values σ_i equal $f(\theta^{(1)}, \dots, \theta^{(m)})$ and the right-hand side of the second check is common to every server, passing all second checks forces $\hat{\sigma}_i = \sigma_i$ for each corrupted server as well, i.e., $\xi_{\sigma,i} = 0$. Substituting the above relation into the honest relation

$$y - \sigma_i = \sum_{j=1}^m \eta^{(j)} \cdot \tau^{(j)}$$

yields the condition

$$\sum_{j=1}^m (\eta^{(j)} \cdot \xi_{\tau,j} + \tau^{(j)} \cdot \xi_{\eta,j} + \xi_{\eta,j} \cdot \xi_{\tau,j}) - \xi_y = 0.$$

We bound the adversary's success probability by considering whether E occurs.

1. **Case $\neg E$:** For a successful forgery in this case, all the first verification checks must pass. By the definition of E , this implies $\xi_{\eta,j} = 0$ for every $j \in [m]$. The condition simplifies to

$$\sum_{j=1}^m \eta^{(j)} \cdot \xi_{\tau,j} - \xi_y = 0.$$

By the same conditioning argument above, conditioned on the adversary's view and random coins, the coefficients $\xi_{\tau,j}$ and ξ_y are fixed while $(\eta^{(j)})_{j \in [m]}$ remains uniform over $\mathbb{F}_p^m$. Since $\xi_y \neq 0$, the displayed expression is a nonzero polynomial of degree at most one, and hence it vanishes with probability at most $1/p$. Averaging over the adversary's view and coins gives

$$\Pr[\text{Exp}(f, T, \text{Adv}) = 1 \wedge \neg E] \leq \frac{1}{p}.$$

2. **Case E :** We trivially have

$$\Pr[\text{Exp}(f, T, \text{Adv}) = 1 \mid E] \leq 1.$$

Thus, the probability of the adversary's success is bounded as follows:

$$\begin{aligned} \Pr[\text{Exp}(f, T, \text{Adv}) = 1] &\leq \Pr[E] + \Pr[\text{Exp}(f, T, \text{Adv}) = 1 \wedge \neg E] \\ &\leq \frac{m}{p} + \frac{1}{p} = \frac{m+1}{p}. \end{aligned}$$

Consequently, for every polynomial $f \in \mathbb{F}_p[X_1, \dots, X_m]$ with $\deg(f) \leq d$, every adversary set $T \in \Delta$, and every unbounded adversary Adv , the success probability in the verifiability experiment of Fig. 1 is at most $\frac{m+1}{p}$.

Efficiency Parameters. The efficiency parameters $(\alpha, \beta, \gamma) = (\alpha' + 3 \log p, \beta', m\beta' + (2m+1) \cdot \log p)$ follow directly from the protocol structure. Each input share

$\text{in}_i^{(j)}$ contains one HSS component $u_i^{(j)}$ and three field elements $(\eta_i^{(j)}, \delta_i^{(j)}, \theta^{(j)})$, establishing a size of $\alpha = \alpha' + 3 \log p$. The output share out_i comprises a single HSS output share, which dictates that $\beta = \beta'$. Finally, each proof share pf_i aggregates m HSS output shares $(\tau_i^{(j)})_{j \in [m]}$ and $2m + 1$ field elements representing σ_i and the auxiliary shares for each input. This combination yields a total size of $\gamma = m\beta' + (2m + 1) \cdot \log p$.

Remark 1. While the adversary success probability bound of $\frac{m+1}{p}$ grows linearly with the number of inputs m , this degradation is a direct consequence of applying the union bound over the m parallel consistency checks required for the individual random masks. From a practical standpoint, because the field size p is typically chosen to be exponentially large (e.g., $p \approx 2^{128}$) in standard cryptographic deployments, the value $\frac{m+1}{p}$ remains cryptographically negligible for any realistically bounded number of clients m . Therefore, this minor theoretical degradation represents a highly favorable and necessary trade-off to achieve compact $O(m)$ proof sizes and support general Q_d adversary structures, bypassing the severe $O(m^d)$ proof size bottlenecks of prior polynomial evaluation schemes.

3.4 Concrete Instantiation Based on CNF Secret Sharing

To demonstrate the real-world applicability of our generic compiler, we present a concrete instantiation using the Conjunctive Normal Form (CNF) secret sharing scheme [18]. CNF secret sharing provides an information-theoretic method for sharing secrets over general adversary structures and serves as a natural baseline for our framework.

Definition 11 (CNF Secret Sharing [18]). *Let Δ be an n -server adversary structure and let Δ^+ denote the collection of all maximal sets within Δ . The SHARE algorithm of Δ -private CNF secret sharing scheme is defined as follows:*

1. *Additively break the secret $s \in \mathbb{F}_p$ into $|\Delta^+|$ shares $\{s_T\}_{T \in \Delta^+}$ such that $s = \sum_{T \in \Delta^+} s_T$.*
2. *The share s_T is distributed to all servers $i \in [n]$ such that $i \notin T$, meaning the server i receives the share s_T if and only if $i \notin T$.*

The inherent structural properties of CNF secret sharing naturally facilitate homomorphic multiplication over shared data. By distributing overlapping subsets of underlying additive shares among participating parties, an algorithm can be constructed that enables the local, non-interactive evaluation of degree-restricted polynomials. This capacity for direct computation establishes the protocol as an additive homomorphic secret sharing scheme, as formalized in the subsequent theorem.

Theorem 2. *Let m, n be positive integers. Let Δ be an n -server adversary structure. If Δ is of type Q_d , the CNF secret sharing scheme over Δ constitutes an additive (m, n, d, Δ) -HSS scheme over $\mathbb{F}_p$.*

Proof. We formalize the additive (m, n, d, Δ) -HSS scheme by explicitly defining the SHARE, EVAL, and DEC algorithms.

For the SHARE algorithm, let the inputs be $x^{(1)}, \dots, x^{(m)} \in \mathbb{F}_p$. Following Definition 11, the algorithm additively shares each input $x^{(j)}$ as $x^{(j)} = \sum_{T \in \Delta^+} x_T^{(j)}$. Each server $i \in [n]$ receives the share components $x_T^{(j)}$ for all T where $i \notin T$.

For the EVAL algorithm, we start with a simpler case, the homomorphic evaluation of a degree- d monomial $\prod_{k=1}^d x^{(j_k)}$. Expanding the product of sums yields a sum of cross-products

$$\prod_{k=1}^d x^{(j_k)} = \sum_{T_1, \dots, T_d \in \Delta^+} \left(\prod_{k=1}^d x_{T_k}^{(j_k)} \right).$$

Because Δ satisfies the Q_d property, the union of any d maximal unauthorized subsets $T_1, \dots, T_d \in \Delta^+$ cannot cover the entire set of servers. Formally, $\bigcup_{k=1}^d T_k \neq [n]$. This property guarantees the existence of at least one server $i \notin \bigcup_{k=1}^d T_k$. Consequently, this capable server i holds all necessary share components $\{x_{T_k}^{(j_k)}\}_{k=1}^d$ and can locally compute the respective cross-product. The evaluation algorithm deterministically assigns each cross-product to exactly one capable server. Each server i then locally sums its assigned cross-products to generate a partial output share out_i . Since any degree- d polynomial is a linear combination of degree- d monomials, the servers can similarly compute linear combinations of these monomial output shares to evaluate arbitrary degree- d polynomials.

For the DEC algorithm, the final output is reconstructed by aggregating the partial output shares. The sum of all output shares $\sum_{i \in [n]} \text{out}_i$ perfectly reconstructs the evaluated polynomial. This structural workflow satisfies all requirements of an additive (m, n, d, Δ) -HSS scheme.

By substituting this CNF-based HSS scheme as the underlying HSS scheme in our generic compiler (detailed in Fig. 2), we directly obtain an information-theoretic $(m, n, d, \Delta, \frac{m+1}{p})$ -VHSS scheme. The compiler automatically augments the basic CNF sharing with the necessary randomized masks and decomposition polynomials, establishing a concrete, verifiable protocol for full polynomial evaluation without relying on any cryptographic assumptions.

3.5 Compatibility of Our Compiler

The compiler presented in Fig. 2 can be applied to an underlying information-theoretic HSS scheme satisfying the following structural properties. These properties provide sufficient conditions for compatibility with our framework.

- **Additive output.** The reconstruction function must be a simple sum of output shares, i.e., $\text{DEC}((\text{out}_i)_{i \in [n]}) = \sum_{i \in [n]} \text{out}_i$. This is essential for the linear consistency check $y - \sigma_i = \sum_{j=1}^m \eta^{(j)} \cdot \tau^{(j)}$ to be well-defined.

- **Local polynomial evaluation.** Each server must be able to evaluate the target polynomial f of degree at most d , as well as the decomposition polynomials q_j , using only its own input shares and without any inter-server communication.
- **Privacy under a Q_d adversary structure.** The underlying HSS scheme must provide information-theoretic privacy against the adversary structure Δ , where Δ is of type Q_d . As established in Proposition 1, the Q_d condition is the feasibility requirement for additive information-theoretic HSS supporting degree- d polynomial evaluation in our setting.

These conditions depend only on the interface provided by the underlying HSS scheme and not on a particular secret-sharing construction. In particular, the CNF-based HSS scheme considered in this work satisfies these conditions. More generally, another HSS construction can be used with our compiler once these properties have been verified.

4 Conclusion

This paper investigates information-theoretic verifiable homomorphic secret sharing (VHSS) for polynomial evaluation over general adversary structures. We construct generic compilers that convert any additive homomorphic secret sharing (HSS) scheme for polynomials into a corresponding VHSS scheme. These compilers significantly expand the application scope of HSS schemes by providing integrity protection for computation results derived from secret-shared data. Constructing error-free VHSS schemes for polynomials over general adversary structures remains a compelling open problem. Furthermore, because our current work fundamentally relies on the honest-client assumption, extending these information-theoretic compilers to tolerate malicious clients that distribute inconsistent or malformed input shares represents a technically non-trivial and fascinating direction for future research. Finally, while previous studies on VHSS establish necessary and sufficient feasibility conditions, our work currently provides a sufficient construction. Establishing the definitive characterization for the existence of information-theoretic VHSS for polynomial evaluation stands as another critical challenge for the field.

Acknowledgments

The authors would like to thank the shepherd Hector Hougaard and the anonymous reviewers for their insightful comments. The work is partially supported by the National Natural Science Foundation of China (Nos. 62372299, 62672322).

References

1. Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 805–817, 2016.

2. Omer Barkol, Yuval Ishai, and Enav Weinreb. On d-multiplicative secret sharing. *Journal of cryptology*, 23(4):580–593, 2010.
3. George Robert Blakley. Safeguarding cryptographic keys. In *Managing requirements knowledge, international workshop on*, pages 313–313. IEEE Computer Society, 1979.
4. Elette Boyle, Niv Gilboa, Matan Hamilis, Yuval Ishai, and Ariel Nof. Fast pcgs for batch-authenticated multiplication triples. In *Advances in Cryptology – CRYPTO 2026, Part VIII*, volume 16807 of *Lecture Notes in Computer Science*, pages 495–526. Springer, 2026.
5. Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under ddh. In *Annual International Cryptology Conference*, pages 509–539. Springer, 2016.
6. Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 1292–1303, 2016.
7. Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. *Cryptology ePrint Archive*, 2017.
8. Chris Brzuska, Geoffroy Couteau, Christoph Egger, Pihla Karanko, and Pierre Meyer. Instantiating the hash-then-evaluate paradigm: strengthening prfs, pcfs, and oprfs. *Cryptography and Communications*, 17(5):1325–1366, 2025.
9. David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, pages 11–19, 1988.
10. Xin Chen and Liang Feng Zhang. Two-server verifiable homomorphic secret sharing for high-degree polynomials. In *International Conference on Information Security*, pages 75–91. Springer, 2020.
11. Xin Chen and Liang Feng Zhang. Publicly verifiable homomorphic secret sharing for polynomial evaluation. *IEEE Transactions on Information Forensics and Security*, 18:4609–4624, 2023.
12. Xin Chen, Liang Feng Zhang, and Jing Liu. Verifiable homomorphic secret sharing for low degree polynomials. *IEEE Transactions on Dependable and Secure Computing*, 20(4):2882–2895, 2022.
13. Ronald Cramer, Ivan Damgård, and Ueli Maurer. General secure multi-party computation from any linear secret-sharing scheme. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 316–334. Springer, 2000.
14. Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Annual cryptology conference*, pages 643–662. Springer, 2012.
15. Reo Eriguchi, Noboru Kunihiro, and Koji Nuida. Multiplicative and verifiably multiplicative secret sharing for multipartite adversary structures. *Designs, Codes and Cryptography*, 91(5):1751–1778, 2023.
16. Daniel Escudero, Cheng Hong, Hongqing Liu, Chaoping Xing, and Chen Yuan. Degree-d reverse multiplication-friendly embeddings: constructions and applications. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 106–138. Springer, 2023.
17. Thibault Feneuil and Matthieu Rivain. Threshold linear secret sharing to the rescue of mpc-in-the-head. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 441–473. Springer, 2023.

18. Mitsuru Ito, Akira Saito, and Takao Nishizeki. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan (Part III: Fundamental Electronic Science)*, 72(9):56–64, 1989.
19. Marcel Keller, Valerio Pastore, and Dragos Rotaru. Overdrive: Making SPDZ great again. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 158–189. Springer, 2018.
20. Russell WF Lai, Giulio Malavolta, and Dominique Schröder. Homomorphic secret sharing for low degree polynomials. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 279–309. Springer, 2018.
21. Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. On the power of sumcheck in secure multiparty computation. In *Public-Key Cryptography – PKC 2026, Part II*, pages 257–287. Springer, 2026.
22. Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. Signatures of correct computation. In *Theory of Cryptography Conference*, pages 222–242. Springer, 2013.
23. Pascal Reiser, Marc Rivinius, Toomas Krips, Sebastian Hasler, and Ralf Küsters. Actively secure polynomial evaluation from shared polynomial encodings. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 3–35. Springer, 2024.
24. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
25. Georgia Tsaloli, Gustavo Banegas, and Aikaterini Mitrokotsa. Practical and provably secure distributed aggregation: Verifiable additive homomorphic secret sharing. *Cryptography*, 4(3):25, 2020.
26. Georgia Tsaloli, Bei Liang, and Aikaterini Mitrokotsa. Verifiable homomorphic secret sharing. In *International Conference on Provable Security*, pages 40–55. Springer, 2018.
27. Georgia Tsaloli and Aikaterini Mitrokotsa. Sum it up: Verifiable additive homomorphic secret sharing. In *International Conference on Information Security and Cryptology*, pages 115–132. Springer, 2019.
28. Ye Xu and Takashi Nishide. Verifiable homomorphic secret sharing for SIMD operations. In *2024 Twelfth International Symposium on Computing and Networking Workshops (CANDARW)*, pages 314–320. IEEE, 2024.
29. Maki Yoshida and Satoshi Obana. Detection of cheaters in non-interactive polynomial evaluation. *Cryptology ePrint Archive*, 2013.
30. Maki Yoshida and Satoshi Obana. Verifiably multiplicative secret sharing. *IEEE Transactions on Information Theory*, 65(5):3233–3245, 2018.
31. Maki Yoshida and Satoshi Obana. Compact verifiably multiplicative secret sharing. In *2020 International Symposium on Information Theory and Its Applications (ISITA)*, pages 437–441. IEEE, 2020.
32. Liang Feng Zhang and Reihaneh Safavi-Naini. Verifiable multi-server private information retrieval. In *International Conference on Applied Cryptography and Network Security*, pages 62–79. Springer, 2014.